

Accelerating k-Nearest Neighbors with GPU-Based Distance Computation

A Practical Comparison of CPU and GPU Approaches for Cross-Validation on Fashion MNIST

Student: Gomery K. Wanjiru
Course: IKT215 – Pattern Recognition
Date: May 2026

Disclaimer on Generative AI

In accordance with UiA’s guidelines, I declare that I used Generative AI tools (GitHub Copilot / Claude) to assist in writing the GPU kNN implementation and structuring this report. The experimental design, analysis, and interpretation are my own work.

Introduction

During the main project for IKT215, I needed to perform a 5-fold cross-validation grid search over 28 hyperparameter combinations for the k-Nearest Neighbors algorithm on the Fashion MNIST dataset. The dataset contains 60,000 training images, each represented as a 784-dimensional vector after flattening.

Using the standard scikit-learn implementation with `joblib` parallelization across 8 CPU cores, this cross-validation took approximately **36 minutes**. The subsequent test prediction on 10,000 samples against the 60,000 training points took an additional 61 seconds. For a project that requires iterating on hyperparameter choices and debugging, this kind of turnaround time is painful.

This led me to ask a simple question: can we offload the distance computation — the core bottleneck of kNN — to the GPU? The answer turned out to be yes, and the results were dramatic enough to warrant this separate report.

The Problem with CPU-Based kNN

The kNN algorithm is conceptually simple: to classify a new sample, compute its distance to all training samples, find the k closest ones, and take a (possibly weighted) majority vote. The computational cost lies almost entirely in the distance computation step, which scales as $O(n \cdot m \cdot d)$ where n is the number of training samples, m is the number of query samples, and d is the dimensionality.

For our 5-fold cross-validation:

- Each fold: 48,000 training samples, 12,000 validation samples, 784 dimensions
- Distance matrix per fold: $12,000 \times 48,000 = 576$ million distance computations
- Total across all folds and combos: $28 \times 5 \times 576\text{M} = 80.6$ billion operations

Scikit-learn uses optimized C code under the hood, and we parallelized across the 28 hyperparameter combinations using 8 CPU workers. But each individual fold still requires computing a massive distance matrix sequentially on a single core. The CPU simply cannot compete with the GPU's massively parallel architecture for this kind of workload.

The GPU Approach

The key insight is that distance computation is essentially a matrix operation. PyTorch provides `torch.cdist(X, Y, p)` which computes the pairwise L_p distance matrix between two sets of vectors, fully utilizing the GPU's thousands of cores.

The implementation is straightforward:

1. Move all training data to GPU memory as a single tensor
2. For each fold, slice the tensor into train/validation portions (no data copy needed)
3. Compute the full distance matrix with `torch.cdist`
4. Use `torch.topk` to find the k nearest neighbors
5. For uniform weighting: `torch.mode` gives the majority vote
6. For distance weighting: accumulate inverse-distance weights per class, take `argmax`

The entire pipeline stays on the GPU — no data transfer between CPU and GPU during the inner loop. The only time data moves back to CPU is for the final accuracy computation.

Memory Considerations

The main constraint is GPU memory. The distance matrix for one fold is $12,000 \times 48,000$ floats = 2.3 GB in float32. On the RTX 4060 Ti (8 GB), this fits but leaves limited headroom. A larger dataset or higher resolution images would require batching the query samples to avoid out-of-memory errors.

For the test prediction (10,000 queries against 60,000 training points), the matrix is $10,000 \times 60,000 = 2.4$ GB — still within budget.

Results

Speed Comparison

Table 1: Runtime comparison of kNN cross-validation approaches.

Method	CV Time	Predict Time	Total
sklearn + joblib (8 cores)	36 min 22s	61.4s	~37 min
GPU (<code>torch.cdist</code>)	7 min 54s	0.0s	~8 min
sklearn GridSearchCV (all cores)	~20 min (est.)	~60s	~21 min

The GPU approach provides a **4.7x speedup** over the original parallelized CPU implementation. The test prediction is effectively instant (the distance matrix for 10k queries is computed in a single GPU kernel call).

The GridSearchCV approach with `n_jobs=-1` is faster than our original joblib implementation because sklearn internally parallelizes across folds rather than across hyperparameter combinations, achieving better load balancing. But it still cannot match the GPU’s raw throughput for distance computation.

Accuracy Comparison

Table 2: Results comparison: CPU vs GPU kNN (best hyperparameters).

	CPU (sklearn)	GPU (PyTorch)
Best k	5	5
Best metric	Manhattan	Manhattan
Best weights	Distance	Distance
CV Accuracy	0.8625	0.8624
Test Accuracy	0.8615	0.8615
Test Precision	0.8627	0.8627
Test Recall	0.8615	0.8615

The results are **identical** in terms of selected hyperparameters and test performance. The 0.0001 difference in CV accuracy is due to floating-point precision: sklearn uses float64 (double precision) while PyTorch on GPU uses float32 (single precision). This difference is negligible and does not affect the model selection outcome.

Per-Class Results

Both implementations produce identical per-class accuracies on the test set:

Table 3: Per-class test accuracy (identical for both implementations).

Class	Accuracy	Class	Accuracy
T-shirt/top	0.8480	Sandal	0.8970
Trouser	0.9700	Shirt	0.5920
Pullover	0.7900	Sneaker	0.9550
Dress	0.8790	Bag	0.9550
Coat	0.7580	Ankle boot	0.9710

Discussion

Why Does the GPU Win?

The GPU excels here because pairwise distance computation is *embarrassingly parallel*. Each element of the $m \times n$ distance matrix can be computed independently — there are no data dependencies between entries. A modern GPU like the RTX 4060 Ti has 4,352 CUDA cores that can all work on different distance computations simultaneously. The CPU, with 8 cores (16 threads), simply cannot match this level of parallelism for this specific workload.

Furthermore, `torch.cdist` is implemented as a series of matrix multiplications and element-wise operations under the hood (using the identity $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a^T b$), which maps perfectly onto the GPU’s tensor cores optimized for matrix multiplication.

Why Not Even Faster?

The GPU approach is “only” 4.7x faster rather than the 100x+ speedups sometimes claimed for GPU computing. The reasons:

1. **Memory bandwidth:** The 2.3 GB distance matrix must be written to GPU memory, then read back for the `topk` operation. At 288 GB/s memory bandwidth on the 4060 Ti, this alone takes measurable time.
2. **Manhattan distance:** Unlike Euclidean distance, L_1 distance cannot be reduced to a matrix multiplication. `torch.cdist` with $p = 1$ must compute element-wise absolute differences, which is less efficient on GPU hardware optimized for multiply-accumulate operations.
3. **Single-batch processing:** We compute the entire fold’s distance matrix at once. Splitting into smaller batches could improve cache utilization but would add kernel launch overhead.

Practical Implications

For a project workflow, the difference between 37 minutes and 8 minutes is significant. It means:

- Faster iteration on hyperparameter grids — you can test more combinations
- Quicker debugging cycles when something goes wrong
- The ability to run the entire pipeline in one sitting without context-switching

For even larger datasets, the GPU advantage would grow further, limited only by GPU memory. Techniques like batched distance computation or approximate nearest neighbors (e.g., FAISS) could push this even further.

Conclusion

By replacing scikit-learn’s CPU-based kNN with a GPU implementation using `torch.cdist` and `torch.topk`, we achieved a 4.7x speedup on the Fashion MNIST 5-fold cross-validation task (from 37 minutes to 8 minutes) while producing numerically identical results. The implementation required minimal code — roughly 30 lines of PyTorch replacing sklearn’s `KNeighborsClassifier` — and demonstrates that even “non-deep-learning” algorithms can benefit substantially from GPU acceleration when their core operation is parallelizable distance computation.

The key takeaway is that the GPU is not just for neural networks. Any algorithm whose bottleneck is a large matrix operation — and kNN’s pairwise distance computation certainly qualifies — can be accelerated by moving it to the GPU. The main limitation is GPU memory, which constrains the maximum distance matrix size that can be computed in a single pass.